

CardioSurv

Individual Task Assignments

Cardiovascular Risk Classification, Treatment Recommendation, Survival Prediction

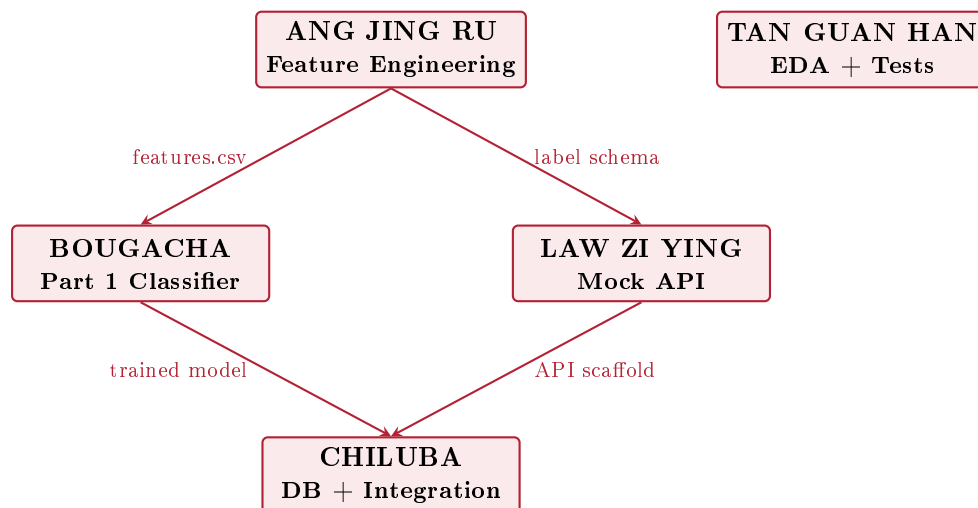
Course: AIT201 Applied Machine Learning, Xiamen University Malaysia

Team of 5

Team Member	Role	Priority
ANG JING RU	Feature Engineering	HIGH PRIORITY
BOUGACHA MOHAMED	Part 1 Risk Classifier	HIGH PRIORITY
LAW ZI YING	Mock FastAPI Backend	HIGH PRIORITY
CHILUBA CHIFUNILO	Database + API Integration	MEDIUM PRIORITY
TAN GUAN HAN	EDA Notebook + Unit Tests	FLEXIBLE TIMING

Dependency Flow

The arrows below show which person's output another person depends on. Anyone with no incoming arrows can start immediately.



Read first: Every task references `docs/schemas.md` and `docs/api_contract.md`. These are the locked Day-1 contracts; treat them as the single source of truth for field names, encodings, and JSON shapes.

1 ANG JING RU — Feature Engineering

Task 1: Build the engineered feature set *HIGH PRIORITY*

Role. Foundation layer. Your output is what two other people need before they can start.

What to build

A Python module that reads `data/processed/cleaned.csv`, adds the three engineered features and the 3-class risk label defined in `docs/schemas.md` §3 and §4, and writes `data/processed/features.csv`.

Files to create or modify

- **CREATE:** `src/features/engineering.py` (currently empty)
- **OUTPUT:** `data/processed/features.csv` (regenerated each run)

Functions to implement

1. `age_bin(age: int) -> str` — returns one of <40, 40-49, 50-59, 60-69, 70+
2. `bp_risk_level(resting_bp: int) -> str` — AHA categories Normal / Elevated / Stage1 / Stage2 / Crisis
3. `hr_stress_index(max_hr: int, age: int) -> float` — formula $\text{max_hr} / (220 - \text{age})$
4. `derive_risk_label(row) -> str` — applies the Low/Medium/High rule from `schemas.md` §4
5. `engineer(df) -> df` — pipeline function that adds all 4 columns
6. `main()` — loads `cleaned.csv`, runs `engineer()`, writes `features.csv`

Snippet to start from

```
def age_bin(age: int) -> str:
    if age < 40: return "<40"
    if age < 50: return "40-49"
    if age < 60: return "50-59"
    if age < 70: return "60-69"
    return "70+"

def hr_stress_index(max_hr: int, age: int) -> float:
    return round(max_hr / (220 - age), 3)
```

Expected output

```
$ python -m src.features.engineering
[features] Input shape: (1421, 12)
[features] Output shape: (1421, 16)
[features] Risk label distribution:
  Low:      746
  Medium:   ~290
  High:     ~385
[features] Wrote data/processed/features.csv
```

Push `features.csv` to GitHub so Bougacha can pull it.

Depends on: Nothing. `cleaned.csv` is already on GitHub.

Blocks: Bougacha (Task 2). Tan Guan Han's `test_features.py`.

2 BOUGACHA MOHAMED — Part 1 Risk Classifier

Task 2: Train and serialise the 3-class risk classifier *HIGH PRIORITY*

Role. First real ML model in the project. Output is consumed by the API layer.

What to build

Train a Random Forest and an XGBoost classifier on `features.csv` targeting the 3-class `RiskCategory` label. Compare both, pick the winner, save it as a `.pkl` file, and wrap inference in a clean `predict(features_dict)` function the API can call with one line.

Files to create or modify

- **CREATE:** `src/models/part1_classifier.py` (currently empty)
- **CREATE:** `notebooks/02_part1_modeling.ipynb` (comparison + rationale)
- **CREATE:** `models/part1_classifier_v1.0.pkl` (binary artefact)

Functions to implement

1. `build_preprocessing_pipeline()` — sklearn Pipeline with `StandardScaler` on numerics + `OneHotEncoder` on categoricals
2. `train_random_forest(X, y)` and `train_xgboost(X, y)`
3. `evaluate(model, X_test, y_test) -> dict` — returns `accuracy`, `f1_macro`, `auc_ovr`, `confusion_matrix`
4. `predict(features_dict: dict) -> dict` — returns the JSON shape from `schemas.md` §4: `risk_category`, `confidence`, `probabilities`, `model_version`
5. `save(model, path) / load(path)` — joblib wrappers
6. `main()` — load → stratified 80/20 split → train both → compare → save winner

Snippet to start from

```
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb

def train_random_forest(X, y):
    clf = RandomForestClassifier(
        n_estimators=300, max_depth=None,
        class_weight="balanced", random_state=42
    )
    clf.fit(X, y)
    return clf

def train_xgboost(X, y):
```

```
clf = xgb.XGBClassifier(
    n_estimators=400, max_depth=6, learning_rate=0.05,
    objective="multi:softprob", num_class=3, random_state=42
)
clf.fit(X, y)
return clf
```

Expected output

```
$ python -m src.models.part1_classifier
[part1] Loading data/processed/features.csv...
[part1] Train: 1136 rows, Test: 285 rows
[part1] Training Random Forest...    acc=0.86    f1_macro=0.84    auc=0.92
[part1] Training XGBoost...          acc=0.89    f1_macro=0.87    auc=0.95
[part1] Winner: XGBoost
[part1] Saved models/part1_classifier_v1.0.pkl
```

Any accuracy above 0.80 is acceptable. The notebook must contain a side-by-side comparison table and a written rationale for the winning model.

Depends on: Jing Ru's `features.csv` (Task 1). If she's not done yet, mock a small CSV with the same columns to start.

Blocks: Chiluba's real API wiring (Task 4 Part B).

3 LAW ZI YING — Mock FastAPI Backend

Task 3: Stand up the API scaffold with mock responses *HIGH PRIORITY*

Role. Backend infrastructure. Lets a frontend (or any consumer) work in parallel with the ML team.

What to build

A working FastAPI app implementing all 5 endpoints from `docs/api_contract.md`, returning **hardcoded JSON**. No real models, no database yet — that's Chiluba's job (Task 4). The Swagger UI at `/docs` must list every endpoint and let you exercise each one from the browser.

Files to create or modify

- **CREATE:** `src/api/main.py` (currently empty)
- **CREATE:** `src/api/schemas.py` (currently empty)

Functions to implement

In `schemas.py`, create Pydantic models for every request/response in `api_contract.md`: `PatientVitalsRequest`, `PredictResponse`, `RecommendRequest`, `RecommendResponse`, `HistoryItem`, `HistoryResponse`, `PatientFullResponse`, `ErrorResponse`.

In `main.py`, implement the 5 endpoints from the contract:

1. GET `/api/v1/health` — returns the `{status, version, model_versions, uptime_seconds}` shape
2. POST `/api/v1/predict` — accepts `PatientVitalsRequest`, returns a realistic-looking mock `PredictResponse`

3. POST `/api/v1/recommend` — alternate SBRT and Medication branch responses based on the `has_arrhythmia` flag
4. GET `/api/v1/history` — returns a list of 5–10 hardcoded historical records
5. GET `/api/v1/patients/{patient_id}` — returns a hardcoded full patient record

Add CORS middleware allowing `http://localhost:5500` and `http://127.0.0.1:5500`.

Snippet to start from

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from src.api.schemas import PatientVitalsRequest, PredictResponse

app = FastAPI(title="CardioSurv API", version="v1")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"])

@app.get("/api/v1/health")
def health():
    return {"status": "ok", "version": "v1",
            "model_versions": {"part1_classifier": "v1.0"},
            "uptime_seconds": 0}

@app.post("/api/v1/predict", response_model=PredictResponse)
def predict(body: PatientVitalsRequest):
    return {"patient_id": "mock-uuid", "risk_category": "Medium",
            "confidence": 0.83,
            "probabilities": {"Low": 0.06, "Medium": 0.83, "High": 0.11},
            "model_version": "part1_classifier_v1.0 MOCK"}
```

Expected output

```
$ uvicorn src.api.main:app --reload
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000
```

Then open `http://localhost:8000/docs` in a browser — every endpoint should be clickable and return valid JSON.

Depends on: Nothing. Only needs `docs/api_contract.md`.

Blocks: No one for the mock. Chiluba (Task 4 Part B) will replace the hardcoded values with real ones.

4 CHILUBA CHIFUNILO — Database + API Integration

Task 4: Persist predictions and wire real models into the API *MEDIUM PRIORITY*

Role. Integration. Where everyone's work converges into a working end-to-end system.

What to build (two parts)

Part A — Database (do first). SQLAlchemy ORM models for the 4 tables defined in `docs/schemas.md` §6, plus a session factory. Use SQLite for local development (file-based, zero setup); the connection string is environment-driven so prod can swap to Postgres without code changes.

Part B — Real wiring (do after Task 2 and Task 3 are partway done). Replace the hard-

coded JSON in `/api/v1/predict` with a real flow: validate input → insert a `patients` row → call Bougacha's `part1_classifier.predict()` → insert a `predictions` row → return the real response. Same treatment for `/api/v1/recommend` using the existing `src/clinical/routing.py`.

Files to create or modify

- **CREATE:** `src/db/models.py` — 4 SQLAlchemy ORM classes
- **CREATE:** `src/db/session.py` — engine, `SessionLocal`, `get_db()` dependency
- **CREATE:** `src/db/seed.py` — insert 10 sample patients for testing
- **MODIFY:** `src/api/main.py` (Law Zi Ying's file) — swap mocks for real calls
- **MODIFY:** `.env.example` — document `DATABASE_URL=sqlite:///cardiosurv.db`

Snippet to start from

```
# src/db/models.py
from sqlalchemy import Column, Integer, String, Numeric, ForeignKey, DateTime,
    JSON
from sqlalchemy.orm import declarative_base, relationship
from datetime import datetime
import uuid

Base = declarative_base()

class Patient(Base):
    __tablename__ = "patients"
    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    created_at = Column(DateTime, default=datetime.utcnow)
    age = Column(Integer, nullable=False)
    sex = Column(String(1), nullable=False)
    # ... other vitals columns from schemas.md sec. 6
    predictions = relationship("Prediction", back_populates="patient")
```

Expected output

Part A:

```
$ python -c "from src.db.session import engine; from src.db.models import Base;
    Base.metadata.create_all(engine)"
$ python -m src.db.seed
[seed] Inserted 10 patients into the database
```

File `cardiosurv.db` appears in the repo root.

Part B:

```
$ curl -X POST http://localhost:8000/api/v1/predict \
    -H "Content-Type: application/json" \
    -d '{"age": 56, "sex": "M", "chest_pain_type": "ATA", ...}'
{"patient_id": "<real-uuid>", "risk_category": "Medium", "confidence": 0.83, ...}
```

A row appears in the `predictions` table that you can inspect with any SQLite client.

Depends on: Nothing for Part A. Part B needs Bougacha's `part1_classifier.py` (Task 2) and Law Zi Ying's `main.py` scaffold (Task 3).

Blocks: No one — you are the integration point.

5 TAN GUAN HAN — EDA Notebook + Unit Tests

Task 5: Exploratory data analysis and clinical-formula tests *FLEXIBLE TIMING*

Role. Quality assurance and report support. Fully independent of everyone else — can start immediately.

What to build (two parts)

Part A — EDA notebook. A Jupyter notebook exploring `cleaned.csv`, generating 4–5 publication-quality PNG figures M6 will use in the report’s Datasets section.

Part B — Unit tests. `pytest` coverage for the already-implemented clinical formulas in `src/clinical/bed.py` and `src/clinical/grace.py`. These currently have zero tests — a real gap before submission.

Files to create or modify

- **CREATE:** `notebooks/01_eda.ipynb`
- **CREATE:** `reports/figures/eda/` folder with 4–5 PNGs
- **MODIFY:** `tests/test_bed.py` — ≥ 6 test functions
- **MODIFY:** `tests/test_grace.py` — ≥ 6 test functions

EDA notebook cells

1. Setup + load `cleaned.csv` and `merged.csv`
2. Shape, `describe()`, `info()`, null counts
3. **Figure:** class balance bar chart (`class_balance.png`)
4. **Figure:** 5-panel histograms of Age, RestingBP, Cholesterol, MaxHR, Oldpeak (`numeric_distributions.png`)
5. **Figure:** stacked-bar categorical breakdowns split by HeartDisease (`categorical_breakdowns.png`)
6. **Figure:** Spearman correlation heatmap (`correlation_heatmap.png`)
7. **Figure:** boxplots of Age per source: Kaggle / Cleveland / Hungarian / VA / Switzerland (`source_comparison.png`)
8. Markdown observations (3–4 short paragraphs M6 can paste directly into the report)

Test coverage required

`test_bed.py` (≥ 6 tests):

- Canonical case: `compute_bed(25, 1)` → BED ≈ 87.5 Gy
- 5-fraction case: `compute_bed(50, 5)` → BED ≈ 100 Gy
- Different α/β : `compute_bed(25, 1, alpha_beta_gy=3)` gives a higher BED
- Negative dose raises `ValueError`
- Zero fractions raises `ValueError`
- EQD2 derivation matches expected value

test_grace.py (≥ 6 tests):

- Low-risk case: 65 y/o, HR 88, SBP 130, Cr 106, Killip I $\rightarrow$ low category
- High-risk case: 80 y/o + cardiac arrest + ST deviation + elevated enzymes $\rightarrow$ high category
- Out-of-range age raises `ValueError`
- Invalid Killip class raises `ValueError`
- Monotonicity: increasing age with everything else fixed increases the score
- Mortality lookup boundaries map to correct risk categories

Snippet to start from

```
# tests/test_bed.py
import pytest
from src.clinical.bed import compute_bed

def test_canonical_25gy_single_fraction():
    r = compute_bed(total_dose_gy=25.0, n_fractions=1)
    assert round(r.bed_gy, 2) == 87.5

def test_negative_dose_raises():
    with pytest.raises(ValueError):
        compute_bed(total_dose_gy=-5.0, n_fractions=1)
```

Expected output

```
$ pytest tests/ -v
tests/test_bed.py::test_canonical_25gy_single_fraction PASSED
tests/test_bed.py::test_negative_dose_raises PASSED
... (12+ tests total)
===== 12 passed in 0.5s =====
```

Depends on: Nothing. `cleaned.csv` and the clinical formulas are already on GitHub.

Blocks: No one. Fully independent.

Coordination Notes

- **Daily 5-minute sync.** One short message in the team chat: “I’m working on X, blocked by Y, will push tonight.” Prevents most integration pain.
- **Pull before you push.** Always `git pull origin main` before starting work *and* before pushing. Otherwise merge conflicts.
- **Push small, push often.** Don’t sit on a finished file for two days. Push as soon as it works.
- **If a dependency isn’t ready, don’t wait.** Mock the input, leave a TODO comment, ship. Wire it up properly once the dependency lands.
- **The contracts are the source of truth.** `docs/schemas.md` and `docs/api_contract.md` — if something is ambiguous, reread them. If they’re wrong, fix them in a PR and tell the team.

Suggested Working Order

Anyone with no incoming arrow on the dependency diagram can start immediately. The recommended sequence, with parallel tracks marked:

1. **Phase 1 (parallel):** Jing Ru (features) | Law Zi Ying (mock API) | Tan Guan Han (EDA)
2. **Phase 2 (parallel):** Bougacha (Part 1 classifier, after `features.csv` lands) | Tan Guan Han (tests)
3. **Phase 3:** Chiluba (DB then real API wiring, after Phase 2 finishes)
4. **Phase 4:** Whole-team integration test — run the 4 example cases from the proposal end-to-end through the live API

End of task assignments. Questions: bring them to the next team sync.